

Student: Djaber Boucharif

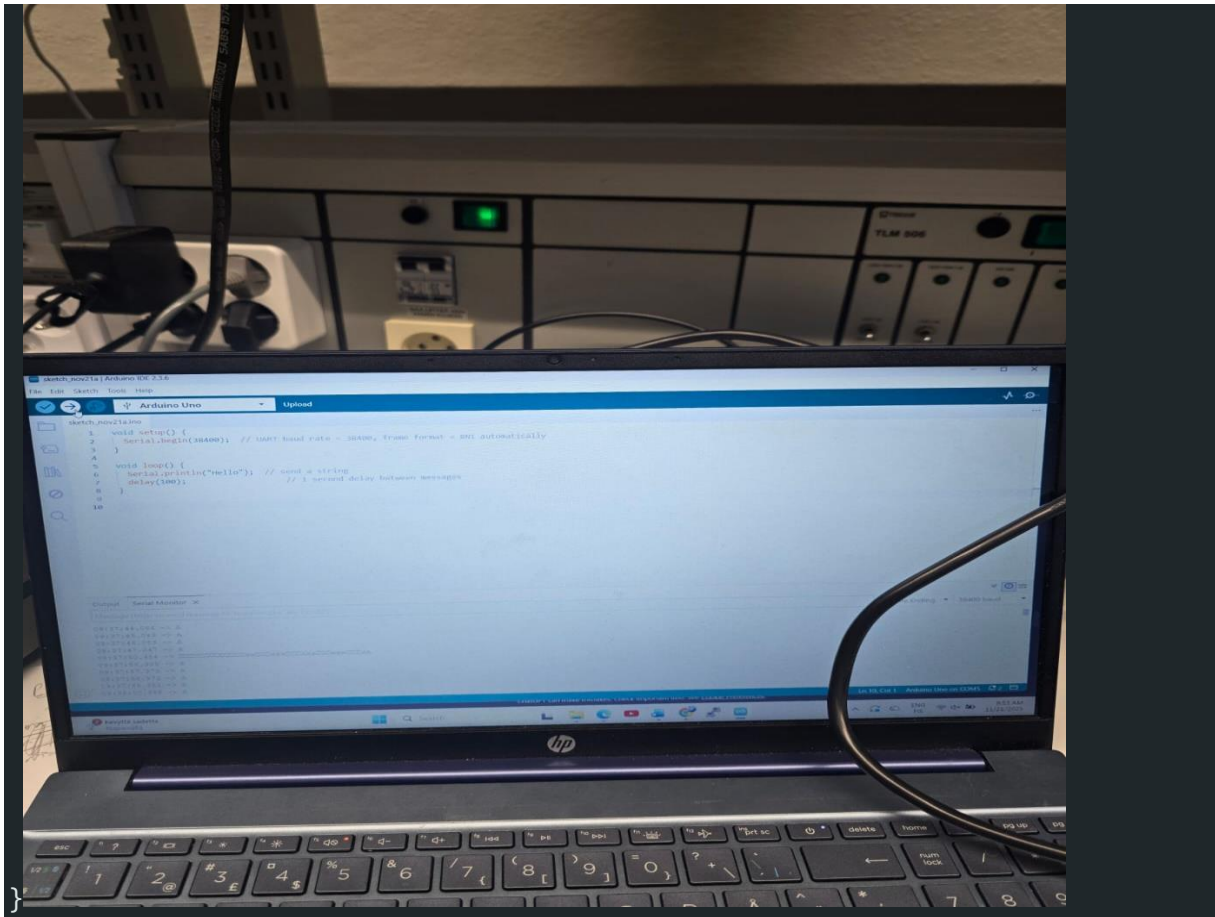
Student Number:2323974

LAB 5: URAT and I2C

Exercise 1:

Code:

```
void setup() {  
  Serial.begin(38400);  
  delay(100);  
}  
  
void loop() {  
  const char* message = "HELLO UART 38400";  
  
  Serial.println(message);  
  
  delay(2000);  
}
```



Exercise 2:

code:

```
#include <avr/io.h>
#include <util/delay.h>

#define F_CPU 16000000UL
#define BAUD 115200
#define UBRR_VALUE ((F_CPU / 16 / BAUD) - 1)

void uart_init() {
    UBRR0H = (unsigned char)(UBRR_VALUE >> 8);
    UBRR0L = (unsigned char)UBRR_VALUE;

    UCSR0C = (1 << USB0) | (1 << UPM01) | (1 << UCSZ01);

    UCSR0B = (1 << TXEN0);
}
```

```
void uart_transmit(unsigned char data) {
    while (!(UCSR0A & (1 << UDRE0)));

    UDR0 = data;
}

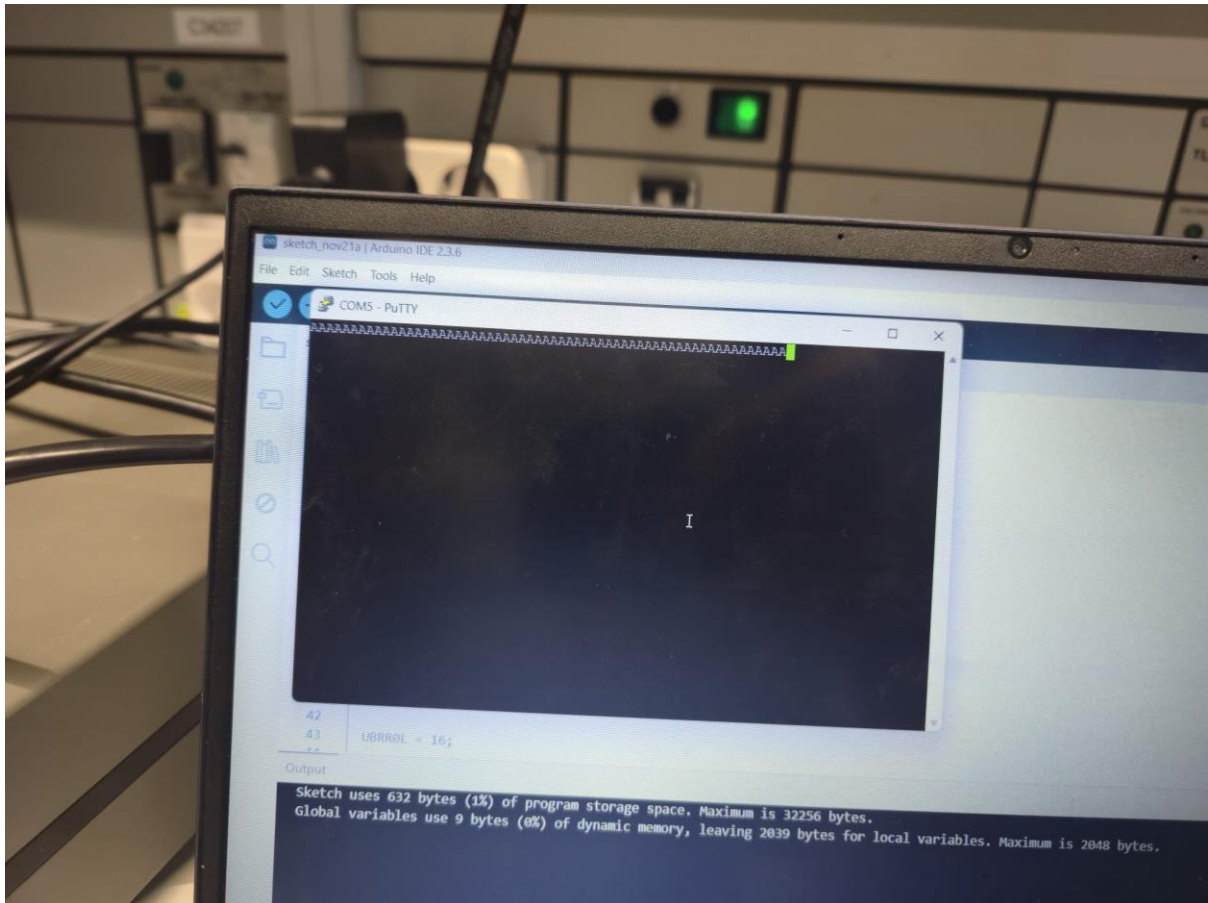
void uart_print(const char *str) {
    for (int i = 0; str[i] != 0; i++) {
        uart_transmit(str[i]);
    }
}

void setup() {
    DDRD |= (1 << PD1);
    uart_init();
}

void loop() {
    const char* message = "BARE METAL 7E2 TEST\r\n";

    uart_print(message);

    _delay_ms(2000);
}
```



Exercise 3:

Code

```
Ultrasonic ultrasonic(12, 13);
int distance;

void setup() {
  Serial.begin(9600);
}

void loop() {

  distance = ultrasonic.read(CM);

  Serial.print("Distance in CM: ");
  Serial.println(distance);

  distance = ultrasonic.read(INC);

  Serial.print("Distance in Inches: ");
  Serial.println(distance);
```



Report and key learnings:

Exercise 1: Simple UART

- **Goal:** Implement and analyze standard UART.
- **Learned:** Configured **38400 baud** and **8N1** format. Used an oscilloscope on **Pin D1** to measure and confirm **bit time** and **half bit time** based on the baud rate.

Exercise 2: Custom UART (Bare Metal)

- **Goal:** Implement custom UART format.
- **Learned:** Used **bare metal** programming to bypass Arduino library limits for non-standard formats (e.g., **7E2**). Configured registers **UCSROB** and **UCSROC** for **115200 baud**. Verified data with **Putty** and timing with the oscilloscope.

Exercise 3: HC-SR04 Connection (I2C)

- **Goal:** Interface with the HC-SR04 sensor using I2C.
- **Learned:** Connected the **4-wire HC-SR04**¹⁰ and used the **Wire.h** library¹¹. Understood the sensor's operation: a **10 us Trig pulse** initiates a burst, and the **Echo pin HIGH time** measures the range¹³¹³¹³¹³. Distance is calculated as $D = (\text{Speed} \cdot \text{Time}) / 2$.

Github: <https://github.com/Djaber-DB/cyber-physical-systems>